UNIT 1 1. Database Basics & Users

- **Database**: Organized collection of structured data. [1]
- **DBMS**: Software to create, manage, and control databases. [1, 2, 3, 4]
- **Database Users**:
  - *DBA*: Manages security, performance, and structure.
  - *Designer*: Defines data structures and relations.
  - *End User*: Queries and updates data (e.g., apps).
  - *Developer*: Writes application programs to interact with DB. [1, 2, 3, 4, 5]

2. Characteristics of DB Systems

- **Self-describing**: Contains data plus description (metadata in catalog).
- **Insulation**: Program-data independence separates structure from code.
- **Data Sharing**: Multiple users access the same data simultaneously.
- **Multi-views**: Users see different filtered subsets of data.
- **Transactions**: Ensures ACID properties (Atomicity, Consistency, Isolation, Durability). [1, 2, 3, 4, 5]

3. Concepts, Architecture & Data Independence

- **Schema**: The overall design/structure (rarely changes). [1, 2, 3]
- **Instance**: Actual data stored at a specific moment (changes constantly). [1, 2, 3]
- **Three-Schema Architecture**:
  - *Internal*: Physical storage structure.
  - *Conceptual*: Global structure, entities, relationships, constraints.
  - *External*: User views. [1, 2, 3, 4, 5]
- **Data Independence**:
  - *Logical*: Changing conceptual schema without changing external views.
  - *Physical*: Changing physical storage without changing conceptual schema. [1, 2, 3, 4]

4. Data Models & Languages

- **Data Model**: Tools describing data, relationships, semantics, and constraints. [1, 2, 3, 4, 5]
- **DBMS Interfaces**: GUI, Command Line, Web forms, API (JDBC/ODBC). [1, 2, 3, 4, 5]
- **DB Languages**:

    - *DDL (Data Definition Language)*: Defines schemas (CREATE, ALTER, DROP).
    - *DML (Data Manipulation Language)*: Handles data (SELECT, INSERT, UPDATE, DELETE).
    - *DCL (Data Control Language)*: Permissions (GRANT, REVOKE). [1, 2, 3, 4, 5]

5. ER & Enhanced ER (EER) Modeling

- **Entity**: Real-world object (Rectangle). [1, 2, 3]
- **Attribute**: Property of an entity (Oval). [1, 2, 3]
- **Relationship**: Association between entities (Diamond).
- **Specialization/Generalization**: Top-down splitting / Bottom-up combining into superclasses and subclasses (IS-A relationship). [1, 2, 3]
- **Aggregation**: Treating a relationship set as a high-level entity. [1]
- **ER to Relational Mapping**:
    - Strong entity \(\rightarrow \) Separate table.
    - Weak entity \(\rightarrow \) Table with owner's Primary Key as Foreign Key.
    - 1:N Relationship \(\rightarrow \) Put Primary Key of "1" side into "N" side as Foreign Key.
    - M:N Relationship \(\rightarrow \) Create a new junction table with both Primary Keys. [1, 2, 3, 4, 5]

6. Views & Indexes

- **View**: Virtual table derived from a SQL query. Does not store physical data.
- **Index**: Data structure (like B-Tree) used to speed up data retrieval. [1, 2, 3, 4, 5]

7. SQL Structure & Constraints

- **Creation**: CREATE TABLE table_name (col data_type constraint); [1, 2]
- **Alteration**: ALTER TABLE table_name ADD/DROP/MODIFY col; [1, 2]
- **Constraints**:
    - PRIMARY KEY: Unique identifier, cannot be NULL (UNIQUE + NOT NULL).
    - FOREIGN KEY: Enforces referential integrity link to another table's PK.
    - UNIQUE: Ensures all values in a column are distinct.
    - NOT NULL: Forbids blank/empty values in a column.
    - CHECK: Validates a condition (e.g., CHECK (age >= 18)).

- IN Operator: Matches value against a literal list (e.g., WHERE status IN ('A', 'B')). [1, 2, 3, 4, 5]

UNIT 2 1. Relational Model Concepts & Constraints

- **Relation**: A table with rows and columns.
- **Tuple**: A single row in a table.
- **Attribute**: A column header.
- **Domain**: Permitted range of values for an attribute.
- **Constraints**:
  - *Domain*: Values must match the data type.
  - *Key*: Minimal set of attributes that uniquely identifies a row.
  - *Entity Integrity*: Primary keys cannot be NULL.
  - *Referential Integrity*: Foreign key must match an existing primary key or be NULL.

2. Relational Algebra vs. Calculus

- **Relational Algebra**: Procedural. Specifies *how* to get data using operators:
  - Select (\(\sigma \)): Filters rows.
  - Project (\(\pi \)): Selects specific columns.
  - Join (\(\Join \)): Combines related tables.
- **Relational Calculus**: Non-procedural. Specifies *what* data to get without listing steps. Uses Tuple Relational Calculus (TRC) or Domain Relational Calculus (DRC).

3. SQL Functions & Set Operations

- **Aggregate Functions**: Summarize multi-row data into one value (SUM, AVG, COUNT, MAX, MIN).
- **Built-in Functions**:
  - *Numeric*: ROUND(), CEIL(), FLOOR(), ABS().
  - *Date*: NOW(), DATE_ADD(), DATEDIFF().
  - *String*: CONCAT(), SUBSTR(), LENGTH(), UPPER(), LOWER().
- **Set Operations**: Combines query results (tables must have identical column structures).
  - UNION: Combines rows, removes duplicates. (UNION ALL keeps duplicates).
  - INTERSECT: Keeps only rows present in both queries.
  - EXCEPT / MINUS: Keeps rows from the first query not present in the second.

4. Subqueries & Clauses

- **Subquery**: A query nested inside another statement.
- **Correlated Subquery**: Inner query depends on the outer query row-by-row (slow processing).
- **Clauses**:
    - ORDER BY: Sorts results (ASC or DESC).
    - GROUP BY: Aggregates rows into summary groups based on matching column values.
    - HAVING: Filters grouped data. **Note**: Used *after* GROUP BY (WHERE filters rows *before* grouping).

5. Joins & Subquery Operators

- **Joins**:
    - *Inner*: Returns rows with matching values in both tables.
    - *Left (Outer)*: Returns all rows from left table, matching rows from right.
    - *Right (Outer)*: Returns all rows from right table, matching rows from left.
    - *Full (Outer)*: Returns all rows when there is a match in either table.
    - *Cross*: Returns Cartesian product (every row combined with every row).
- **Operators**:
    - EXISTS: True if subquery returns at least one row.
    - ANY / SOME: True if comparison matches *at least one* value in the subquery list.
    - ALL: True if comparison matches *every* value in the subquery list.

6. Views & Transaction Control (TCL)

- **View Types**:
    - *Simple View*: Created from one table, no aggregate functions, can be updated.
    - *Complex View*: Created from multiple tables, contains joins/aggregates, cannot be easily updated.
- **TCL Commands**:
    - COMMIT: Saves all pending data changes permanently to the database.
    - ROLLBACK: Erases all changes since the last commit/checkpoint.
    - SAVEPOINT: Sets a temporary marker to which you can roll back partially.

UNIT 3 1. Normalization & Functional Dependencies (FD)

- **Normalization**: Process of decomposing tables to eliminate data redundancy and anomalies (Insert, Update, Delete).
- **Functional Dependency ($X \rightarrow Y$)**: X uniquely determines Y. If two rows have the same X, they must have the same Y.
- **Decomposition Properties**:
    - *Lossless Join*: Joining decomposed tables reconstructs the original table exactly (no extra/missing rows). Formula: $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$.
    - *Dependency Preserving*: All original FDs can be checked within the decomposed tables without joining them.

2. Normal Forms (1NF to 5NF, DKNF)

- **1NF**: Atomic values only. No repeating groups or multi-valued attributes.
- **2NF**: In 1NF + No **partial dependency** (no non-prime attribute depends on a *subset* of a candidate key).
- **3NF**: In 2NF + No **transitive dependency** (no non-prime attribute depends on another non-prime attribute). *Rule*: For $X \rightarrow Y$, X must be a super key or Y must be a prime attribute.
- **BCNF**: Stronger than 3NF. *Rule*: For $X \rightarrow Y$, X **must** be a super key.
- **4NF**: In BCNF + Eliminates **multivalued dependencies** ($X \rightarrow\rightarrow Y$).
- **5NF**: In 4NF + Eliminates **join dependencies** (table cannot be decomposed further without losing information upon joining).
- **DKNF (Domain-Key)**: Ultimate normal form. Every constraint is a direct consequence of domain constraints and key constraints.

3. Transactions & Schedules

- **ACID Properties**:
    - *Atomicity*: All-or-nothing execution.
    - *Consistency*: Database moves from one valid state to another.
    - *Isolation*: Concurrent transactions do not interfere with each other.
    - *Durability*: Committed changes are permanent, even after a system crash.
- **Transaction States**: Active $\rightarrow$ Partially Committed $\rightarrow$ Committed. If failed: Active/Partially Committed $\rightarrow$ Failed $\rightarrow$ Aborted.
- **Schedules**: Sequence of execution of instructions from concurrent transactions.
    - *Serial*: One transaction finishes completely before the next starts (always consistent).

    - *Serializable*: Non-serial schedule that produces the same result as a serial schedule.
- **Recoverable Schedule**: If \(T_{2}\) reads data written by \(T_{1}\), then \(T_{1}\) must commit *before* \(T_{2}\) commits. Prevents dirty read issues.

4. Concurrency Control & Deadlocks

- **Locking Techniques**:
    - *Shared Lock (\(S\))*: For reading data. Multiple transactions can hold \(S\) simultaneously.
    - *Exclusive Lock (\(X\))*: For writing data. Only one transaction can hold \(X\).
    - *Two-Phase Locking (2PL)*: Guarantees serializability. Has a Growing Phase (acquiring locks) and a Shrinking Phase (releasing locks). No locks can be acquired once releasing begins.
- **Timestamp Ordering (TO)**: Orders transactions based on system timestamps. Older transactions get priority. If a conflict occurs, the younger transaction is rolled back.
- **Granularity**: Size of data items locked (Database \(\rightarrow \) Table \(\rightarrow \) Page \(\rightarrow \) Row). Coarse granularity (large items) means low concurrency but low lock overhead. Fine granularity means high concurrency but high lock overhead.
- **Deadlock**: Two transactions waiting indefinitely for locks held by each other.
    - *Detection*: Maintained via a **Wait-For Graph (WFG)**. Cycles indicate a deadlock.
    - *Recovery*: Roll back one of the deadlocked transactions (the victim).

5. Recovery Techniques

- **Log-Based Recovery**: Uses a history file (log) containing undo/redo records.
    - *Deferred Update*: Writes to database only *after* commit. Needs **Redo** only.
    - *Immediate Update*: Writes to database mid-transaction. Needs **Undo** and **Redo**.
- **Checkpoints**: Points in the log where all modifications are flushed to disk. Speeds up recovery by avoiding scans of the entire log.
- **Catastrophic Failures**: Recovery requires restoring the database from a remote backup and re-applying the log file transactions up to the failure point.

6. Database Programming

- **Control Structures**: Code blocks handling logic flow (IF-THEN-ELSE, LOOP, WHILE).

- **Exception Handling**: Code blocks designed to catch and handle runtime errors without crashing.
- **Stored Procedures**: Precompiled SQL code blocks stored on the server for reusability and performance.
- **Triggers**: Special stored programs that execute **automatically** in response to specific events (e.g., BEFORE/AFTER INSERT, UPDATE, DELETE).

UNIT 4 1. Secondary Storage & File Structures

- **Secondary Storage Devices**: Magnetic disks (HDDs) and Solid State Drives (SSDs). Data is read/written in fixed-size blocks (pages), not individual bytes.
- **File Operations**: Open, Find/Read, Insert, Delete, Update, Close, and Scan.
- **Heap Files (Unordered)**: New records are placed at the end of the file.
    - *Insert*: Very fast (\(O(1)\)).
    - *Search/Delete*: Slow (\(O(N)\) linear scan needed).
- **Sorted Files (Ordered)**: Records are sequentially ordered by a specific "search key" attribute.
    - *Search*: Fast (\(O(\log N)\) using binary search).
    - *Insert/Delete*: Slow (\(O(N)\) because remaining records must be physically shifted).

2. Hashing

- **Static Hashing**: A hash function maps a search key directly to a fixed array of disk blocks (buckets). Cannot dynamically shrink or grow.
- **Dynamic Hashing**: Buckets grow and shrink dynamically as data size changes (e.g., Extendible Hashing using a directory, Linear Hashing using splitting rules). Prevents performance degradation from overflow buckets.

3. Indexing Techniques

- **Single-Level Indexes**: An auxiliary file containing (key, pointer) pairs, sorted by the key.
    - *Primary Index*: Ordered by a primary key on a physically ordered file (one entry per block/sparse).
    - *Clustering Index*: Ordered by a non-key field on a physically ordered file (one entry per distinct value).
    - *Secondary Index*: Ordered on an unordered file. Must be *dense* (one entry per individual row/record).
- **Multi-Level Indexes**: Created when the index file itself becomes too large for memory. A primary index is built on top of the original index file (creates an inner/outer index hierarchy).

4. Tree-Based Indexing

- **B-Tree**: A self-balancing search tree.
    - *Structure*: Every node contains both search keys **and** direct pointers to actual data records.
    - *Downside*: Leaves are not linked; range queries require expensive tree traversals.
- **B+ Tree**: The standard database indexing structure.
    - *Structure*: Internal nodes *only* store routing keys and child pointers. **All data pointers and actual keys reside strictly in the leaf nodes**.
    - *Advantage*: Leaf nodes are structurally linked sequentially as a doubly linked list. This allows incredibly fast sequential range scans (`WHERE age BETWEEN 20 AND 30`).

5. Advanced Database Paradigms

- **OODBMS (Object-Oriented DBMS)**: Integrates object-oriented programming concepts directly with database storage.
    - *Key Concepts*: Stores data as complex Objects (instead of flat rows). Uses Object Identifiers (OID) instead of keys, and supports encapsulation, inheritance, and methods directly inside the DB.
- **DDBMS (Distributed DBMS)**: A single logical database split physically across multiple geographical computing nodes connected via a network.
    - *Data Fragmentation*: Splitting a table into subsets (Horizontal: splitting rows; Vertical: splitting columns).
    - *Data Replication*: Copying identical data across multiple nodes to increase availability, fault tolerance, and local read speeds.

Now that you have all four units mapped out, let me know if you would like me to:

- Show a quick **comparison table of B-Trees vs B+ Trees** for easy memorization.
- Give you a **text diagram of horizontal vs vertical fragmentation** to visualize for the exam.
- Provide a **set of high-yield practice questions** combining all the units you've listed.

  You said: convert all this in word document with minimum spacing ike zero size and give me in that format